

Ballmaschine

Madina hat eine heißluftbasierte Ballmaschine erfunden, die auf dem Prinzip basiert, dass ein(em) gewisser/n Femboy eben heiß ist. Der interne Aufbau der Maschine ist wie folgt:

- Die Maschine besteht aus N **Knoten**, die von 0 bis $N - 1$ indiziert sind. Der Knoten $N - 1$ wird als **Wurzel** bezeichnet.
- Für jeden Knoten u mit $0 \leq u < N - 1$ ist der **Elternknoten** des Knotens u ein Knoten $P[u]$ mit $P[u] > u$, und der Knoten u ist ein **Kind** von $P[u]$. Die Wurzel hat keinen Elternknoten. Ein Knoten ohne Kinder wird als **Blatt** bezeichnet.

Der Wert von N ist **nicht** bekannt, genauso wie die Elternknoten $P[u]$ von keinem Knoten u bekannt sind. Allerdings ist die Anzahl M der Blätter bekannt, sowie dass die Blätter die Indizes $0, 1, \dots, M - 1$ haben. Deine Aufgabe ist es, die interne Struktur der Maschine zu ermitteln.

Jeder Knoten der Maschine kann höchstens einen **Ball** enthalten, und jeder Ball hat einen **Wert**, der eine nicht-negative ganze Zahl ist. Wir sagen, dass ein Knoten den Wert x hat, wenn er einen Ball mit dem Wert x enthält. Ein Knoten ist **belegt**, wenn er einen Ball enthält; andernfalls ist er **frei**. Zu Beginn ist jeder Knoten frei.

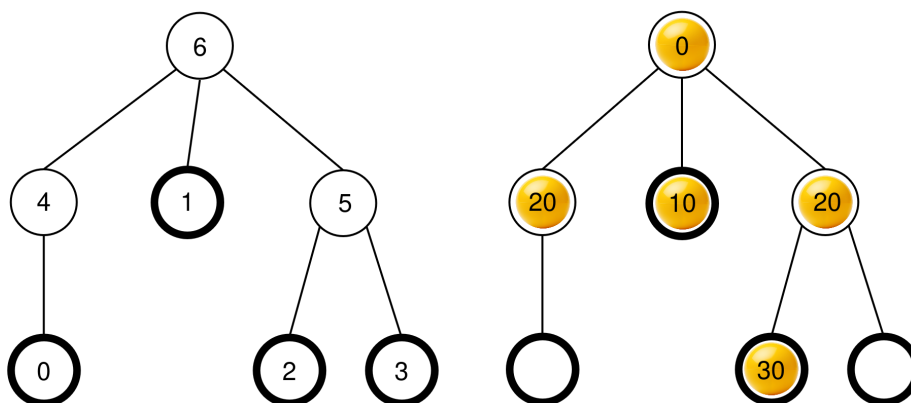
Du kannst zwei Arten von Operationen durchführen:

- insert(U, X):** Schreibe den Wert X auf eine neue Kugel und versuche, sie am Blattknoten U einzufügen.
 - Ist der Blattknoten U belegt, gibt die Operation `false` zurück, ohne die Kugel einzufügen.
 - Ist der Blattknoten U frei, wird die Kugel am Knoten U platziert. Anschließend bewegt sich der Ball wiederholt zum Elternknoten, solange dieser frei ist. Die Bewegung endet, wenn der Ball die Wurzel oder einen Knoten erreicht, dessen Elternknoten belegt ist. Die Operation gibt `true` zurück.
- collect():** Wenn die Wurzel frei ist (was bedeutet, dass die Maschine leer ist), gibt `collect` ein leeres Array zurück. Andernfalls sammelt die unten beschriebene rekursive Funktion `traverse` die Werte aller Bälle, die sich derzeit in der Maschine befinden, in einem Array S . Zu Beginn ist das Array S leer und `traverse( $N - 1$ )` wird aufgerufen.

```
traverse(u):
    Hänge den Wert des Balls auf Knoten u an das Ende von S an.
    Sei c[u] die Liste der Kinder belegten Kinder von u.
    Sortiere c[u] in nicht absteigender Reihenfolge nach dem Wert der Bälle bei diesen Knoten (wenn mehrere Kno
    for each v in c[u]:
        traverse(v)
```

Nach dem Aufruf von `collect` werden **alle Bälle aus der Maschine entfernt**, und `collect` gibt S zurück.

Betrachten wir beispielsweise eine Maschine mit $N = 7$ Knoten und dem Eltern-Array $P = [4, 6, 5, 5, 6, 6]$. Die Abbildung links zeigt die leere Maschine mit den Knotenindizes, während die Abbildung rechts eine mögliche Anordnung der Kugeln nach einer Abfolge von `insert`-Operationen darstellt (diese Abfolge ist im Abschnitt Beispiel angegeben).



Wenn `collect()` aufgerufen wird, ist der Wurzelknoten (Knoten 6) belegt, sodass S als $S = []$ initialisiert und `traverse(6)` aufgerufen wird.

traverse(6): Der Knoten 6 hat den Wert 0; durch Anhängen an S ergibt sich $S = [0]$. Die Kinder des Knotens 6 sind 4, 1 und 5, die alle belegt sind. Ihre Werte lauten 20, 10 bzw. 20. Die Sortierung in nicht absteigender Reihenfolge ergibt $c[6] = [1, 5, 4]$ (beachte, dass auch $c[6] = [1, 4, 5]$ gültig wäre, da die Knoten 4 und 5 denselben Wert haben). Das Verfahren ruft dann `traverse` für jeden Knoten in $c[6]$ in dieser Reihenfolge auf.

- **traverse(1)**: Der Knoten 1 hat den Wert 10; durch Anhängen an S ergibt sich $S = [0, 10]$. Der Knoten 1 hat keine besetzten Kinder, daher endet dieser Aufruf.
- **traverse(5)**: Der Knoten 5 hat den Wert 20; durch Anhängen an S ergibt sich $S = [0, 10, 20]$. Der Knoten 5 hat ein besetztes Kind, den Knoten 2, daher ist $c[5] = [2]$ und die Funktion ruft **traverse(2)** auf.
 - **traverse(2)**: Der Knoten 2 hat den Wert 30; durch Anhängen an S ergibt sich $S = [0, 10, 20, 30]$. Der Knoten 2 hat keine besetzten Kinder, daher endet dieser Aufruf.
 - Die Ausführung kehrt zu **traverse(5)** zurück, das alle Kinder in $c[5]$ verarbeitet hat, sodass der Aufruf beendet wird.
- **traverse(4)**: Der Knoten 4 hat den Wert 20; durch Anhängen an S ergibt sich $S = [0, 10, 20, 30, 20]$. Der Knoten 4 hat keine belegten Kinder, daher endet dieser Aufruf.

Alle Aufrufe sind abgeschlossen und es gibt keine weiteren Knoten mehr zu verarbeiten. Schließlich wird jeder Ball aus der Maschine entfernt und **collect** gibt das Array $S = [0, 10, 20, 30, 20]$ zurück.

Deine Aufgabe ist es, die Anzahl der Knoten N zu ermitteln und ein Array von Elternknoten $R = [R[0], R[1], \dots, R[N-2]]$ auszugeben, welches die Struktur der Maschine beschreibt, jedoch mit einer möglicherweise abweichenden Beschriftung der Knoten, die weder Blattknoten noch die Wurzel sind. Formal gilt ein zurückgegebenes Array R als korrekt, wenn es möglich ist, jedem Knoten u der Maschine ein unterschiedliches Label $L[u]$ mit $0 \leq L[u] < N$ zuzuweisen, sodass gilt:

- $L[u] = u$ für alle $0 \leq u < M$ und $u = N-1$, und
- $L[P[u]] = R[L[u]]$ für alle $0 \leq u < N-1$.

Du musst **nicht** sicherstellen, dass $R[u] > u$ gilt. Die Maschine **muss leer sein**, wenn dein Programm die Lösung zurückgibt.

Sei K die Anzahl der durchgeführten **collect**-Operationen und B der maximale Wert, der bei allen **insert** Aufrufen auf eine Kugel geschrieben wurde. Der Wert $C = K + B$ **darf höchstens 1000 sein**. Deine Punktzahl hängt in einigen Teilaufgaben vom Wert von C ab.

Details zur Implementierung

Dein Programm muss folgende Funktion implementieren:

```
std::vector<int> find_structure(int M)
```

- M : die Anzahl der Blätter in der Maschine.
- Die Funktion wird für jeden Testfall genau einmal aufgerufen.
- Die Funktion muss ein korrektes Array $R = [R[0], R[1], \dots, R[N-2]]$ der Elternknoten mit Länge $N-1$ zurückgeben.

Um mit der Maschine zu interagieren, kann dein Programm die folgenden zwei Funktionen aufrufen.

Die erste Funktion lautet:

```
bool insert(int U, int X)
```

- U : Der Index des Blattknotens, in den der Ball eingefügt werden soll. Die Bedingung $0 \leq U < M$ muss erfüllt sein.
- X : der ganzzahlige Wert des Balls. Die Bedingung $0 \leq X \leq 1000$ muss erfüllt sein.
- Diese Funktion gibt **true** zurück, wenn der Ball erfolgreich eingefügt wurde, oder **false**, wenn das Blatt U bereits belegt war.
- Diese Funktion kann höchstens 500 000 Mal aufgerufen werden.

Die zweite Funktion lautet:

```
std::vector<int> collect()
```

- Sammelt die Werte aller eingelegten Bälle ein, leert die Maschine und gibt das gesammelte Array S zurück.

Wenn zu irgendeinem Zeitpunkt während der Ausführung deines Programms der Wert von C den Wert 1000 überschreitet, erhält deine Lösung das Urteil `Output isn't correct: Too many resources used`.

Die Struktur der Maschine ist festgelegt, bevor **find_structure** aufgerufen wird. Insbesondere ist der Grader **deterministisch** in dem Sinne, dass wenn dieser zweimal ausgeführt wird und jede Ausführung die gleiche Sequenz an Operationen durchführt, dann auch die Rückgabewerte der **collect** Funktion in beiden Ausführungen identisch sind.

Beschränkungen

- $2 \leq N \leq 1000$
- $1 \leq M \leq 200$
- $M < N$

Teilaufgaben

Teilaufgabe	Punktzahl	Weitere Beschränkungen
1	5	Die Wurzel hat genau M Kinder.
2	10	$M \leq 3$
3	25	$N \leq 200, M \leq 45$
4	60	Keine weiteren Beschränkungen.

In den Teilaufgaben 3 und 4 hängt deine Punktzahl wie folgt vom Wert von C ab.

Teilaufgabe 3 (25 Punkte)

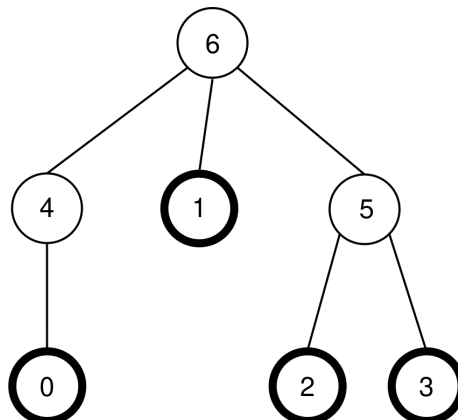
Grenzen	Punktzahl
$1000 < C$	0
$45 < C \leq 1000$	13
$C \leq 45$	25

Teilaufgabe 4 (60 Punkte)

Grenzwerte	Punktzahl
$1000 < C$	0
$200 < C \leq 1000$	7
$71 < C \leq 200$	$47 - \frac{C}{5}$
$44 < C \leq 71$	$104 - C$
$C \leq 44$	60

Beispiel

Betrachten wir dieselbe Maschine wie in der Aufgabenbeschreibung, also $N = 7$, $M = 4$ und $P = [4, 6, 5, 5, 6, 6]$. Die Maschine ist erneut in der folgenden Abbildung dargestellt:



Der Grader führt den folgenden Aufruf aus:

```
find_structure(4)
```

Dein Programm kann die folgende Abfolge von Aufrufen ausführen:

1. `insert(0, 0)`: Das Blatt 0 ist frei, daher wird ein Ball mit dem Wert 0 am Blatt 0 platziert. Der Knoten $P[0] = 4$ ist frei, daher bewegt sich der Ball zum Knoten 4. Der Knoten $P[4] = 6$ ist frei, daher bewegt sich der Ball zum Knoten 6. Da der Knoten 6 die Wurzel ist, wird die Bewegung beendet. Der Aufruf gibt `true` zurück.
2. `insert(3, 20)`: Das Blatt 3 ist frei, daher wird ein Ball mit dem Wert 20 am Blatt 3 platziert. Der Knoten $P[3] = 5$ ist frei, daher bewegt sich der Ball zum Knoten 5. Da $P[5] = 6$ belegt ist, wird die Bewegung beendet. Der Aufruf gibt `true` zurück.
3. `insert(1, 10)`: Das Blatt 1 ist frei, daher wird ein Ball mit dem Wert 10 am Blatt 1 platziert. Da $P[1] = 6$ belegt ist, verbleibt der Ball am Knoten 1. Der Aufruf gibt `true` zurück.

4. `insert(2, 30)`: Das Blatt 2 ist frei, daher wird ein Ball mit dem Wert 30 am Blatt 2 abgelegt. Da $P[2] = 5$ belegt ist, verbleibt der Ball am Knoten 2. Der Aufruf gibt `true` zurück.
5. `insert(1, 25)`: Das Blatt 1 ist belegt, daher wird der Ball nicht am Blatt 1 abgelegt. Der Aufruf gibt `false` zurück.
6. `insert(0, 20)`: Das Blatt 0 ist frei, daher wird ein Ball mit dem Wert 20 am Blatt 0 abgelegt. Der Knoten $P[0] = 4$ ist frei, daher bewegt sich der Ball zum Knoten 4. Da $P[4] = 6$ belegt ist, stoppt die Bewegung. Der Aufruf gibt `true` zurück.

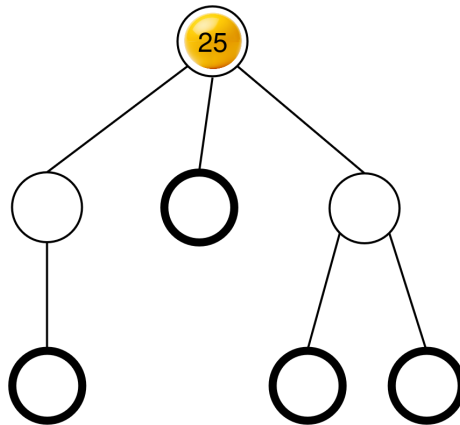
Die resultierende Anordnung der Bälle wurde bereits in der Aufgabenbeschreibung dargestellt.

Als Nächstes kann dein Programm folgendes aufrufen:

```
collect()
```

Die Ausführung dieses Aufrufs wurde bereits in der Aufgabenbeschreibung erläutert, daher gibt dieser Aufruf $S = [0, 10, 20, 30, 20]$ zurück. Danach ist die Maschine wieder leer.

Als Nächstes kann die Funktion `insert(2,25)` aufrufen. Der Blattknoten 2 ist frei, daher wird eine Kugel mit dem Wert 25 am Blattknoten 2 abgelegt. Diese Kugel bewegt sich dann zum Knoten 5 und anschließend zum Knoten 6, wo sie zum Stillstand kommt. Der Aufruf gibt `true` zurück. Die resultierende Anordnung der Bälle ist in der folgenden Abbildung dargestellt.



Schließlich kann dein Programm `collect()` aufrufen, was $S = [25]$ zurückgibt und die Maschine leert.

Es gibt zwei korrekte Arrays, die `find_structure` zurückgeben kann: $R = P = [4, 6, 5, 5, 6, 6]$ (was der Beschriftung $L = [0, 1, 2, 3, 4, 5, 6]$ entspricht) und $R = [5, 6, 4, 4, 6, 6]$ (was der Beschriftung $L = [0, 1, 2, 3, 5, 4, 6]$ entspricht). In diesem Beispiel gilt: $K = 2$ und $B = 30$, also $C = 32$.

Beispiel-Grader

Eingabeformat:

```
N M
P[0] P[1] P[2] ... P[N-2]
```

Ausgabeformat:

```
K B C
Q
R[0] R[1] R[2] ... R[Q-1]
```

Hier ist Q die Länge des Arrays R , das von `find_structure` zurückgegeben wird.